JR Validated Environment

User Guide for Design Verification

Version 1.0 | Date: 2026-03-16 | Audience: Design Engineers

1. How to Use This Guide

This guide helps you choose the right script and run it correctly. You do not need a statistics background — you need to know your test, your requirement, and where you are in the process. Answer three questions to find your section:

Question 1 — What kind of data does my test produce?

- Pass / fail (the unit either passes or fails, no measurement value) → Section 3.1 or 4.5
- A measured value (force, dimension, pressure, torque, …) → Sections 3.2 – 4.4

Question 2 — Where am I in the process?

- Planning a test → Section 3
- Analysing data or writing a report → Section 4

Question 3 — What do I need to show or find out? → Use the Quick Reference table in Section 2.

Tip: If you are running an analysis for the first time, start with Section 6 (First-Time Setup) to make sure the scripts are available on your machine.

2. Quick Reference Table

Find the row that matches what you are trying to do, then go to the section listed.

I want to…	Script	Section
I need to know how many units to test (pass/fail)	jrc_ss_discrete	3.1
I need to know how many units to test (measured values)	jrc_ss_attr	3.2
I want to check if my planned N is large enough	jrc_ss_attr_check	3.3
I want to verify my pilot sigma estimate is reliable	jrc_ss_sigma	3.4
I need to show two methods / designs are equivalent	jrc_ss_equivalence	3.5
I need to compare two groups with paired measurements	jrc_ss_paired	3.6
I need to qualify my measurement system (GRR)	jrc_ss_gauge_rr	3.7
I need to plan a fatigue or reliability test	jrc_ss_fatigue	3.8
I want to check if my data is normally distributed	jrc_normality	4.1
I want to check for outliers in my dataset	jrc_outliers	4.2
I want to check process capability (Cpk)	jrc_capability	4.3

I want to report a verification result (measured values)	jrc_verify_attr / jrc_ss_attr_ci	4.4
I want to report a verification result (pass/fail)	jrc_ss_discrete_ci	4.5
I want to compare two measurement methods	jrc_bland_altman	4.6
I want a summary of my dataset	jrc_descriptive	4.7
I want to analyse fatigue or lifetime data	jrc_weibull	4.8

3. Test Planning

Use the scripts in this section before you run your test, to determine the required sample size or study design. Always run these scripts using the same proportion and confidence values you intend to use for the final verification report.

3.1 jrc_ss_discrete — Sample size for pass/fail tests

When do I use this?

Use this when your test produces a simple pass or fail result (no measurement value). The script outputs a table of required sample sizes for 0 to 10 allowed failures, so you can evaluate the trade-off between sample size and acceptance criterion. The most common case in medical device verification is zero allowed failures (f = 0).

What do I need?

Argument	What to provide
proportion	The minimum fraction of the population that must pass (e.g. "0.99" for 99% reliability).
confidence	The statistical confidence level (e.g. "0.95" for 95%).

Example

```
jrc_ss_discrete "0.99" "0.95"
```

What to look for in the output

Find the row for your planned acceptance criterion (f = number of allowed failures). The 'n' column is the minimum sample size. For zero failures, 99% reliability at 95% confidence requires n = 299 — in practice round up to 300. If the required n is too large, consider whether f = 1 or f = 2 is acceptable for your risk classification.

3.2 jrc_ss_attr — Sample size for tests with measured values

When do I use this?

Use this when your test measures a value (force, dimension, pressure, etc.) and your requirement specifies one or two spec limits. You will need a small pilot dataset (typically 5–10 samples) collected under representative conditions. The script uses a tolerance interval approach: it finds the minimum sample size such that you can claim, with the specified confidence, that the specified proportion of the population falls within your spec limits. Run jrc_ss_sigma first (Section 3.4) to verify your pilot dataset is large enough.

What do I need?

Argument	What to provide
proportion	The minimum fraction of the population that must meet the spec

	(e.g. "0.99" for 99%).
confidence	The statistical confidence level (e.g. "0.95" for 95%).
file_path	Full path to your pilot dataset CSV file.
column_name	Name of the column containing the measurements.
spec1	Lower spec limit (LSL), or "-" if there is no lower limit.
spec2	Upper spec limit (USL), or "-" if there is no upper limit.

Example

```
jrc_ss_attr "0.99" "0.95" "~/data/pilot.csv" "ForceN" "15" "40"
```

What to look for in the output

The script reports the minimum required sample size N. If your data is not normally distributed, the script attempts a Box-Cox transformation automatically and reports the result in original units. Use the recommended N (or a slightly larger number for practical reasons) as your verification sample size.

3.3 jrc_ss_attr_check — Check if your planned N is large enough

When do I use this?

Use this as a quick check before finalising your test plan. If you already have a target sample size in mind (e.g. from a historical study or a practical constraint), this script tells you whether that N is statistically sufficient. It is faster than jrc_ss_attr because it answers a yes/no question rather than searching for the minimum N.

What do I need?

Argument	What to provide
proportion	Same value as used in jrc_ss_attr (e.g. "0.99").
confidence	Same value as used in jrc_ss_attr (e.g. "0.95").
file_path	Full path to your pilot dataset CSV file.
column_name	Name of the column containing the measurements.
spec1	Lower spec limit, or "-".
spec2	Upper spec limit, or "-".
planned_N	The sample size you are planning to use (positive integer).

Example

```
jrc_ss_attr_check "0.99" "0.95" "~/data/pilot.csv" "ForceN" "15" "40" "35"
```

What to look for in the output

The script prints PASS if your planned N is sufficient, or FAIL with the minimum required N. If it fails, run jrc_ss_attr to find the exact minimum and adjust your test plan accordingly.

3.4 jrc_ss_sigma — Is my pilot dataset large enough?

When do I use this?

Use this before jrc_ss_attr to verify that your pilot dataset gives a reliable estimate of the process standard deviation. The sample size calculated by jrc_ss_attr is only trustworthy if the sigma estimate is stable. The 'precision' argument controls how sensitive the check is: 1.5 is a sensible default for most verification studies.

What do I need?

Argument	What to provide
precision	How precisely sigma must be estimated. Use "1.5" as a starting point (detects a 1.5-sigma shift). Lower values are more demanding and require more pilot samples.
spec1	Lower spec limit, or "-".
spec2	Upper spec limit, or "-".

Example

```
jrc_ss_sigma "1.5" "15" "40"
```

What to look for in the output

The script reports the minimum pilot sample size needed for your spec limits and precision setting. If your pilot dataset is smaller than this number, collect more pilot samples before running jrc_ss_attr.

3.5 jrc_ss_equivalence — Sample size for an equivalence study

When do I use this?

Use this when you need to demonstrate that two designs, methods, or fixtures produce results that are equivalent within a predefined margin (delta). This is common when introducing a new measurement fixture or manufacturing process that must be shown to be interchangeable with an existing one. Each sample is measured on both methods (paired design). The delta value must be defined before data collection — it cannot be set after seeing the data.

What do I need?

Argument	What to provide
delta	The maximum acceptable difference between the two methods, in the same units as your measurements (e.g. "0.5" for ±0.5 N·m).
sd	Estimated standard deviation of the differences between methods, from a pilot study or historical data (same units as delta).
sides	"1" for a one-sided equivalence test; "2" for two-sided (most common).

Example

```
jrc_ss_equivalence "0.5" "0.3" "2"
```

What to look for in the output

The script outputs a table of required sample sizes (number of pairs) for a range of power levels. Use the row corresponding to 80% power as a minimum, or 90% for higher assurance. Round up to the next practical number.

3.6 jrc_ss_paired — Sample size for a paired comparison

When do I use this?

Use this when you need to detect a specific difference between two groups or time points, using a paired design (the same units measured under both conditions). Unlike jrc_ss_equivalence, this script is for detecting a difference rather than demonstrating equivalence. Common applications: before/after studies, environmental stress testing, or two-arm comparisons.

What do I need?

Argument	What to provide
delta	The minimum difference you need to be able to detect (same units as your measurements).
sd	Estimated standard deviation of the paired differences.
sides	"1" for a one-sided test; "2" for two-sided.

Example

```
jrc_ss_paired "2.0" "3.5" "2"
```

What to look for in the output

The script outputs a sample size table for a range of power levels. Choose the row matching your required power (80% is the conventional minimum; 90% or 95% for critical tests). Note that n is the number of pairs, not the number of individual observations.

3.7 jrc_ss_gauge_rr — Measurement system qualification (GRR)

When do I use this?

Use this to determine the sample size for a Gauge Repeatability and Reproducibility (GRR) study, or to check whether your measurement system is adequate given an existing GRR result. The script follows AIAG guidelines. A measurement system with %GRR > 30% is generally considered inadequate for verification purposes and should be improved before proceeding.

What do I need?

Argument	What to provide
grr	Target %GRR as a decimal (e.g. "0.10" for 10% GRR).
type	"sigma" if you are working in process sigma units; "tolerance" if you are working relative to the tolerance width.
sigma_or_tolerance	Process sigma (if type = sigma) or tolerance width (if type = tolerance), in measurement units.

Example

```
jrc_ss_gauge_rr "0.10" "tolerance" "25"
```

What to look for in the output

The script reports the number of parts, operators, and replicates needed to achieve the target %GRR, and the NDC (Number of Distinct Categories). NDC ≥ 5 is required for the measurement system to be useful for process control. NDC ≥ 2 is the minimum for verification pass/fail decisions.

3.8 jrc_ss_fatigue — Sample size for fatigue / reliability tests

When do I use this?

Use this when your test involves cyclic loading, wear, or time-to-failure, and you need to determine sample size based on a required reliability level. The script uses a Weibull model. You need to provide the Weibull shape parameter (beta) from prior data or literature, and the acceleration factor if the test is accelerated. For beta, a value of 2–4 is typical for wear-out failure modes; beta = 1 means constant failure rate (exponential).

What do I need?

Argument	What to provide
reliability	Required reliability at the end of design life (e.g. "0.90" for 90%).
confidence	Statistical confidence level (e.g. "0.95" for 95%).
shape	Weibull shape parameter beta from prior data or literature.
af	Acceleration factor for accelerated tests. Use "1" for non-accelerated (real-time) tests.

Example

```
jrc_ss_fatigue "0.90" "0.95" "2.5" "1"
```

What to look for in the output

The script reports the minimum sample size N. If the test is accelerated (af > 1), the sample size accounts for the acceleration. The output also shows the B-life (time at which a specified percentage of the population is expected to have failed) for context. An f = 0 assumption (zero allowed failures) is used by default, consistent with FDA expectations for life-critical devices.

4. Data Analysis

Use the scripts in this section after collecting your data, to check data quality, compute the verification result, and summarise findings for the report.

Recommended sequence for continuous (measured) data: Step 1: jrc_descriptive — get an overview of the dataset. Step 2: jrc_normality — check distribution. Step 3: jrc_outliers — investigate unusual observations. Step 4: jrc_capability — check Cpk against your spec. Step 5: jrc_verify_attr — report the formal verification result. Use the same proportion and confidence values throughout.

4.1 jrc_normality — Check if data is normally distributed

When do I use this?

Use this to determine whether your dataset follows a normal distribution. This matters because the tolerance interval methods used by jrc_ss_attr and jrc_verify_attr assume normality (or a transformable-to-normal distribution). The script runs three complementary tests and reports a plain-English conclusion. If the data is not normal, the script attempts a Box-Cox transformation — the downstream scripts handle this automatically.

What do I need?

Argument	What to provide
file_path	Full path to your CSV file.
column_name	Name of the column to test.

Example

```
jrc_normality "~/data/verif_force.csv" "ForceN"
```

What to look for in the output

Look for the conclusion line at the end of the output: 'Data appears normal' or 'Data does not appear normal'. If not normal, check whether Box-Cox transformation succeeds — if it does, jrc_verify_attr will handle the

transformation automatically. If neither normality nor transformation is achieved, consult a statistician before proceeding with the tolerance interval approach.

4.2 jrc_outliers — Check for outliers

When do I use this?

Use this after checking normality to identify observations that are statistically unusual. Outliers are not automatically removed — the script flags them so you can investigate their root cause. An outlier should only be excluded from analysis if there is a documented, assignable cause (e.g. operator error, equipment malfunction). Never remove outliers simply because they are inconvenient.

What do I need?

Argument	What to provide
file_path	Full path to your CSV file.
column_name	Name of the column to check.

Example

```
jrc_outliers "~/data/verif_force.csv" "ForceN"
```

What to look for in the output

The script reports results from two methods: Grubbs (best for normally distributed data) and IQR (distribution-free). If both methods flag the same observation, treat it as a strong signal. If only one method flags it, investigate but do not remove without cause. The output shows which row IDs (from the 'id' column) are flagged.

4.3 jrc_capability — Process capability (Cpk)

When do I use this?

Use this to check how well your process is centred and how much margin exists relative to the spec limits. Cpk is a single number: the higher, the better. A Cpk of 1.33 means the process spread fits comfortably within the spec with margin on both sides. This is the conventional minimum for a capable process. Cpk < 1.0 means the process is generating out-of-spec units.

What do I need?

Argument	What to provide
file_path	Full path to your CSV file.
column_name	Name of the measurement column.
spec1	Lower spec limit (LSL), or "-".
spec2	Upper spec limit (USL), or "-".

Example

```
jrc_capability "~/data/verif_force.csv" "ForceN" "15" "40"
```

What to look for in the output

Key values to report: Cpk (centring-aware index) and its 95% CI. If only one spec limit applies, only Cpk is meaningful (Cp requires both limits). A Cpk ≥ 1.33 with the lower bound of the 95% CI above 1.0 is a strong

result. If Cpk is low but all units pass the spec, the process is capable but not robust — consider a larger verification sample.

4.4 jrc_verify_attr and jrc_ss_attr_ci — Report a verification result (measured values)

When do I use these?

Use these two scripts together to produce the formal verification result for a continuous measurement requirement. They answer complementary questions:

- jrc_verify_attr: Does my dataset demonstrate that the required proportion of the population falls within the spec limits? Reports the tolerance interval bounds and a PASS / FAIL conclusion.
- jrc_ss_attr_ci: Given my dataset, what is the highest proportion I can claim at the specified confidence? Use this to report the demonstrated reliability level (e.g. '≥99.2% at 95% confidence').

Use the same proportion and confidence values that were used during sample size planning.

jrc_verify_attr — What do I need?

Argument	What to provide
proportion	Minimum fraction that must meet the spec (e.g. "0.99").
confidence	Confidence level (e.g. "0.95").
file_path	Full path to your verification dataset CSV.
column_name	Name of the measurement column.
spec1	Lower spec limit, or "-".
spec2	Upper spec limit, or "-".

Example (jrc_verify_attr)

```
jrc_verify_attr "0.99" "0.95" "~/data/verif_force.csv" "ForceN" "15" "40"
```

Example (jrc_ss_attr_ci)

```
jrc_ss_attr_ci "0.95" "~/data/verif_force.csv" "ForceN" "15" "40"
```

What to look for in the output

jrc_verify_attr reports the tolerance interval bounds (in original measurement units) and a clear PASS or FAIL against your spec limits. A histogram PNG is saved alongside your input CSV — include this in your verification report. jrc_ss_attr_ci reports the maximum demonstrated proportion (e.g. ≥99.4%) — use this number in the report summary statement.

4.5 jrc_ss_discrete_ci — Report a verification result (pass/fail)

When do I use this?

Use this after a pass/fail verification test to calculate the demonstrated reliability at your chosen confidence level. This is the pass/fail counterpart to jrc_ss_attr_ci.

What do I need?

Argument	What to provide
proportion	The required reliability level (e.g. "0.99").
n	Total number of units tested.

f	Number of failures observed.

Example

```
jrc_ss_discrete_ci "0.99" "300" "0"
```

What to look for in the output

The script reports the confidence level achieved given your n and f. If you tested 300 units with 0 failures against a 99% reliability requirement, the output will confirm that 95%+ confidence is achieved (PASS). If failures occurred, the script shows the confidence level that was achieved — compare this to your acceptance criterion.

4.6 jrc_bland_altman — Compare two measurement methods

When do I use this?

Use this when you want to assess whether two measurement methods agree well enough to be used interchangeably, or to characterise the bias between a test method and a reference method. Each sample is measured by both methods. The script computes the bias (mean difference), the limits of agreement (LoA), and saves a Bland-Altman plot. Note: jrc_bland_altman takes two separate CSV files — one per method.

What do I need?

Argument	What to provide
file1	Path to CSV file for method 1 (reference method or Device A).
col1	Column name in file1 containing the measurements.
file2	Path to CSV file for method 2 (test method or Device B).
col2	Column name in file2 containing the measurements.

Important: Both files must have the same number of rows, and rows are matched by position. Row 1 of file1 must correspond to the same sample as row 1 of file2. The difference is computed as method 2 minus method 1.

Example

```
jrc_bland_altman "~/data/fixture_a.csv" "TorqueNm" "~/data/fixture_b.csv"
"TorqueNm"
```

What to look for in the output

Key values: Bias (mean difference between methods) and its 95% CI, and the Limits of Agreement (LoA = bias ± 1.96 × SD of differences). The LoA define the range within which 95% of differences between the methods are expected to fall. Compare the LoA to your predefined acceptance criterion. A proportional bias test is also reported — if significant, the agreement between methods changes across the measurement range, which may need further investigation.

4.7 jrc_descriptive — Dataset summary

When do I use this?

Use this as a first look at any dataset before running other analyses. It provides a complete descriptive summary including mean, median, standard deviation, percentiles, skewness, and a 95% confidence interval on the mean. It is especially useful for spotting data entry errors, unexpected ranges, or high variability before investing time in more detailed analysis.

What do I need?

Argument	What to provide
file_path	Full path to your CSV file.
column_name	Name of the column to summarise.

Example

```
jrc_descriptive "~/data/verif_force.csv" "ForceN"
```

What to look for in the output

Check the min and max against your expected measurement range — values outside this range may indicate data entry errors or equipment issues. Check the CV (coefficient of variation) to understand variability relative to the mean: a CV above 20–30% typically warrants investigation before proceeding to tolerance interval analysis. Check skewness — values above ±1 suggest the data may not be normally distributed.

4.8 jrc_weibull — Analyse fatigue or lifetime data

When do I use this?

Use this to analyse time-to-failure or cycle-to-failure data from a fatigue or durability test. The script fits a two-parameter Weibull distribution and reports the shape (beta) and scale (eta) parameters, the B10/B50 life (the time at which 10%/50% of units are expected to have failed), and reliability at a given life. Right-censored data (units that did not fail before the test ended) are handled correctly — mark these with status = 0 in your CSV.

What do I need?

Argument	What to provide
file_path	Full path to your CSV file.
time_col	Column name containing the observed time or cycles (failure time, or test end time for surviving units).
status_col	Column name containing the event indicator: 1 = failed, 0 = did not fail (censored).

Example

```
jrc_weibull "~/data/fatigue_results.csv" "cycles" "status"
```

What to look for in the output

Key values: shape (beta) and scale (eta) with confidence intervals, B10 life (time by which 10% of units are expected to have failed), and demonstrated reliability at your design life. Beta > 1 indicates a wear-out failure mode (increasing failure rate over time). Beta < 1 indicates early-life failures (decreasing failure rate). Beta ≈ 1 indicates a random failure mode.

5. Worked Examples

The following three examples walk through realistic end-to-end verification scenarios using generic data. Commands and outputs are illustrative — your actual numbers will differ.

5.1 Example 1: Dimensional verification — sealing force (continuous)

Scenario

A catheter connector must produce a sealing force between 15 N and 40 N. The verification plan requires demonstrating that 99% of connectors meet this requirement at 95% confidence. A pilot dataset of 8 connectors has been collected under representative production conditions.

Step 1 — Verify the pilot dataset is large enough

```
jrc_ss_sigma "1.5" "15" "40"
```

Output indicates a minimum of 7 pilot samples are needed for the given spec width and precision setting. With 8 samples, the pilot is sufficient. Proceed.

Step 2 — Check normality of pilot data

```
jrc_normality "~/data/pilot_force.csv" "ForceN"
```

Shapiro-Wilk p = 0.42, Anderson-Darling p = 0.38. Conclusion: Data appears normal. No transformation needed.

Step 3 — Determine verification sample size

```
jrc_ss_attr "0.99" "0.95" "~/data/pilot_force.csv" "ForceN" "15" "40"
```

Pilot mean = 27.3 N, pilot SD = 3.8 N. Minimum sample size: N = 32. The test plan uses N = 35 to provide additional margin.

Step 4 — Confirm planned N is sufficient (optional pre-check)

```
jrc_ss_attr_check "0.99" "0.95" "~/data/pilot_force.csv" "ForceN" "15" "40"
"35"
```

Output: PASS — planned N = 35 meets the requirement. Test proceeds.

Step 5 — Check for outliers in verification data

```
jrc_outliers "~/data/verif_force.csv" "ForceN"
```

Grubbs test: no outliers detected. IQR method: no outliers detected. All 35 observations are retained for analysis.

Step 6 — Check process capability

```
jrc_capability "~/data/verif_force.csv" "ForceN" "15" "40"
```

Verification dataset: mean = 27.1 N, SD = 3.6 N. Cpk = 1.49 (95% CI: 1.18 – 1.80). The process is well-centred and capable.

Step 7 — Report the verification result

```
jrc_verify_attr "0.99" "0.95" "~/data/verif_force.csv" "ForceN" "15" "40"
```

Tolerance interval: [17.9 N, 36.3 N] ⊂ [15 N, 40 N]. Result: PASS. A histogram PNG is saved alongside the CSV.

```
jrc_ss_attr_ci "0.95" "~/data/verif_force.csv" "ForceN" "15" "40"
```

Demonstrated proportion: ≥ 99.4% at 95% confidence.

Report statement: "At 95% confidence, ≥ 99.4% of the population is demonstrated to fall within the specification of 15 – 40 N. The tolerance interval [17.9 N, 36.3 N] lies fully within the specification. Verification PASSED."

5.2 Example 2: Leak test verification (pass/fail)

Scenario

A sealed pouch must withstand a 30 kPa pressure hold for 30 seconds with no visible leak. The acceptance criterion is: demonstrate ≥ 99% reliability at 95% confidence with zero failures allowed. The test is destructive — all units are pressure-tested to failure or pass.

Step 1 — Determine sample size

```
jrc_ss_discrete "0.99" "0.95"
```

The script outputs a table. For f = 0 (zero failures), the required sample size is n = 299. The test plan rounds up to n = 300 for practicality. The table also shows that f = 1 would require n = 473 — not practical, so the zero-failure criterion is retained.

Step 2 — Run the test

300 pouches are tested. Result: 0 failures. All 300 units pass the pressure hold.

Step 3 — Report the verification result

```
jrc_ss_discrete_ci "0.99" "300" "0"
```

Output: At 99% reliability, the achieved confidence is 95.0%. Acceptance criterion: ≥ 95% confidence. Result: PASS.

Report statement: "300 pouches were tested to the 30 kPa / 30 s pressure hold criterion. 0 failures were observed. By the exact binomial method, ≥ 99% reliability is demonstrated at 95% confidence. Verification PASSED."

5.3 Example 3: Equivalence of two torque measurement fixtures

Scenario

A new torque measurement fixture is being introduced to replace a legacy fixture. The requirement is that the two fixtures must agree within ± 0.5 N·m. A pilot study estimates the standard deviation of differences between fixtures at approximately 0.30 N·m. A two-sided equivalence study is planned.

Step 1 — Determine sample size for the equivalence study

```
jrc_ss_equivalence "0.5" "0.3" "2"
```

At 80% power: n = 18 pairs. At 90% power: n = 24 pairs. The study uses n = 25 pairs to allow for one unusable sample.

Step 2 — Run the study

25 representative units are each measured on both the legacy fixture (A) and the new fixture (B). Measurements are stored in two CSV files: fixture_a.csv and fixture_b.csv, with rows matched by sample ID.

Step 3 — Analyse agreement between fixtures

```
jrc_bland_altman "~/data/fixture_a.csv" "TorqueNm" "~/data/fixture_b.csv"
"TorqueNm"
```

Results:

Statistic	Value
n (pairs)	25
Bias (B – A)	+0.05 N·m (95% CI: −0.07 to +0.17)
SD of differences	0.24 N·m
Limits of Agreement	−0.42 to +0.52 N·m (95% CI of bounds: −0.57 to +0.67)
Proportional bias test	r = 0.12, p = 0.56 — not significant

Step 4 — Interpret the result

The bias of +0.05 N·m is small and its 95% CI includes zero — the two fixtures produce essentially unbiased results on average. The upper limit of agreement (+0.52 N·m) marginally exceeds the ± 0.5 N·m acceptance criterion. This is a borderline result that requires careful interpretation.

Borderline result — what to do: The point estimate of the LoA (+0.52 N·m) exceeds the ± 0.5 N·m limit by 0.02 N·m. Before concluding, consider: (1) Is the acceptance criterion a hard regulatory limit or an engineering target? (2) The 95% CI on the upper LoA extends to +0.67 N·m — this represents the statistical uncertainty. (3) If the criterion is a hard limit, document the margin and discuss with your project statistician and quality team before signing off. A common approach is to accept the result with a documented justification when the exceedance is within measurement uncertainty.

Report statement (example with documented justification): "25 pairs of measurements were taken on both fixtures. The mean bias is +0.05 N·m (95% CI: −0.07 to +0.17), indicating negligible systematic difference. The 95% Limits of Agreement are −0.42 to +0.52 N·m. The upper LoA marginally exceeds the ± 0.5 N·m acceptance criterion by 0.02 N·m. This exceedance is within the measurement uncertainty of the study (SD = 0.24 N·m) and is considered not practically significant. The fixtures are considered equivalent for verification purposes. [Deviation documented as DEV-XXX.]"

6. Utility Commands

Three utility commands are available to all users. They are in the bin/ folder and are on your PATH automatically after running setup_jr_path.zsh.

6.1 jrrun — run any R or Python script

jrrun is the engine behind every JR wrapper script. You can also call it directly to run any R or Python script in the validated environment — not just the community scripts in the R/ or Python/ folders.

Syntax: jrrun <script.[R|py]> [arguments...]

```
Bare filename:    jrrun myscript.R data.csv
                  Looks in R/ or Python/ subfolder automatically.

Relative path:    jrrun ./myscript.R data.csv
                  Resolves from your current working directory.

Absolute path:    jrrun /Users/you/projects/analysis.R data.csv

Per-script help:  jrrun myscript.R --help
```

Before running the script, jrrun verifies the project integrity (SHA256 check), rebuilds the R or Python environment if package versions have changed, and logs the run to ~/.jrscript/<PROJECT_ID>/run.log. Scripts outside the JR folder can be run through jrrun and will execute in the fully validated environment.

Important: R scripts check at startup that they were launched through jrrun. Running a script directly with Rscript will fail immediately with an error. Always use jrrun or a wrapper script.

6.2 jr_versions — check installed versions

jr_versions prints a version report for the current machine: R version, Python version, and every pinned package with its installed version and a pass/fail indicator.

Syntax: jr_versions

Use jr_versions to confirm your environment is correctly installed after first-time setup, after a package update, or when troubleshooting an unexpected result. The report is also useful during an audit — it shows exactly which package versions are running on a specific machine at a specific point in time.

6.3 jr_uninstall — remove your local environment

jr_uninstall removes your personal JR environment components from the current machine. Use it when you are decommissioning a machine, resolving a corrupted installation, or handing back equipment.

Syntax: jr_uninstall

The script shows exactly what will be removed before asking for confirmation. Type 'yes' to proceed. The following are removed:

- R package library (~/.renv/<PROJECT_ID>/renv/library/)
- Python virtual environment (~/.venvs/<PROJECT_ID>/)
- Run log (~/.jrscript/<PROJECT_ID>/run.log)
- PATH entry from ~/.zprofile

The renv package cache (~/.renv/<PROJECT_ID>/cache/) is deliberately preserved. This means if you reinstall later, packages are restored from the local cache without re-downloading from the Dropbox repository. Validation evidence files in ~/.jrscript/<PROJECT_ID>/validation/ are also preserved.

After uninstalling: Your environment will be rebuilt automatically the next time you run any JR script. No further action is required.

7. Contributing a Script

If you have written an R or Python script that could be useful to your team or to the wider community, there are three ways to share it.

7.1 Personal use — no admin required

You can run any script you have written directly in the validated environment using jrrun. No changes to the project are needed.

```
jrrun ./myscript.R mydata.csv
```

Your script runs with the same pinned R or Python packages and the same integrity-checked infrastructure as the community scripts.

7.2 Team contribution — ask your administrator

If your script is useful to the wider team and you want it to become an official community script, ask your administrator to add it. You provide:

- The finished R or Python script.
- Help text — what the script does, what arguments it takes, and a usage example.

The administrator uses admin_scaffold_R or admin_scaffold_Python to create the wrapper and help file, adds any required packages, regenerates the integrity file, and runs the full OQ test suite to confirm everything passes before the script is available to the team.

Note on revalidation: Adding a community script is a change to the validated configuration. The administrator must confirm that the OQ test suite passes in full before the script is used in a regulated context.

7.3 Public contribution — GitHub

If your script could be useful to organisations beyond your own team, you are welcome to contribute it to the public repository at github.com/ubrowz/jr-validated-env.

Open a GitHub issue first, describing what the script does, when an engineer would use it, what inputs and outputs it has, and any new packages it requires. If the proposal is approved, submit a pull request containing the script, wrapper, help file, and OQ tests.

Accepted scripts are included in a future minor release with a CHANGELOG entry and updated validation documents. See CONTRIBUTING.md in the repository root for the full submission requirements.

8. First-Time Setup

Before you can run any script by name, you need to run the setup script once. This tells your Mac where to find the JR scripts. You only need to do this once per machine.

Step 1 — Run the setup script

Open a Terminal window, navigate to the JR scripts folder, and run:

```
zsh setup_jr_path.zsh
```

You will see a confirmation message: '✅ PATH updated successfully.' If you see '✅ PATH already configured', the setup was already done on this machine.

Step 2 — Open a new Terminal window

The PATH change takes effect in new Terminal windows only. Close the current window and open a fresh one.

Step 3 — Verify the setup

Run any script with the --help flag:

```
jrc_descriptive --help
```

You should see the help text for the script. If you see 'command not found', check that step 1 completed successfully, and that you opened a new Terminal window.

Note: If the scripts are still not found after following these steps, contact your system administrator. Do not attempt to modify the PATH manually.

9. Glossary

Statistical terms used in this guide, explained in plain English.

Term	Plain-English Definition
Bias	The average difference between two measurement methods or between a measurement and the true value. A bias of zero means the methods agree on average. A non-zero bias means one method consistently reads higher or lower than the other.
Box-Cox transformation	A mathematical operation that converts non-normal data into approximately normal data. Applied automatically by jrc_ss_attr, jrc_verify_attr, and related scripts when the data is not normally distributed. Results are always reported back in original (untransformed) units.
Confidence level	The probability that the statistical claim is correct. A 95% confidence level means that if you repeated the study many times, 95% of the resulting intervals would contain the true value. In regulatory contexts, 95% is the conventional minimum.
Cpk	A process capability index that measures how centred and tight your process is relative to the spec limits. Cpk ≥ 1.33 is a commonly accepted threshold for a capable process. Cpk = 1.0 means the process just fits within the spec with no margin. Cpk < 1.0 means out-of-spec units are being produced.
Equivalence margin (delta)	The maximum difference between two methods that is considered acceptable. Must be defined before data collection based on clinical or engineering judgement — never set after seeing the data.
f (number of failures)	In pass/fail testing, the number of units that fail the test. The acceptance criterion specifies the maximum allowed f. f = 0 (zero failures) is common for life-critical devices.
Limits of Agreement (LoA)	In a Bland-Altman analysis, the range within which 95% of differences between two methods are expected to fall (bias ± 1.96 × SD of differences). Compare the LoA to your predefined equivalence margin to judge agreement.
NDC (Number of Distinct Categories)	In a GRR study, the number of groups the measurement system can reliably distinguish. NDC ≥ 5 is required for process control; NDC ≥ 2 for pass/fail decisions.
Proportion	The fraction of the population that meets the requirement. A proportion of 0.99 means 99% of units are expected to pass. Used in tolerance interval calculations and pass/fail reliability claims.
Reliability	In the context of pass/fail testing (jrc_ss_discrete), reliability is the same as proportion — the fraction of the population expected to pass the test.
Tolerance interval	A statistical interval calculated from sample data that, with a

	specified confidence, contains at least a specified proportion of the population. Used by jrc_verify_attr to demonstrate that a process meets its spec limits.
Weibull shape parameter (beta)	A number that describes how the failure rate changes over time. Beta > 1: failure rate increases over time (wear-out). Beta = 1: constant failure rate. Beta < 1: decreasing failure rate (early-life failures). Typical values for mechanical wear-out: 2 – 5.
%GRR	In a Gauge R&R study, the percentage of the process variation (or tolerance) that is due to measurement system variation. %GRR < 10%: excellent. 10–30%: acceptable. > 30%: inadequate.